

# A Heterogeneous Computing Architectures for Sequence-to-Graph Alignment

Anonymous Author(s)

## Abstract

The rapid development of high-throughput sequencing and population genomics has made it increasingly difficult for traditional linear reference models to capture the full genetic diversity of populations. Graph-based reference models offer a more comprehensive approach by modeling multiple variations, but they introduce significant computational challenges in sequence-to-graph alignment, such as large search spaces and scheduling difficulties, especially in long-read sequencing scenarios.

To address these issues, we propose an innovative heterogeneous computational architecture for sequence-to-graph alignment. This system integrates a reinforcement learning-based scheduling accelerator (RLSA) designed to optimize resource utilization, load balancing, and adaptive task scheduling. RLSA introduces a context cost matrix (CCM) for dynamic scheduling, enabling the system to adapt to real-time changes in task context and execution cost. By leveraging hardware-friendly graph representations and dynamic task decomposition, this architecture improves efficiency across various datasets. Experimental results demonstrate that the heterogeneous system integrating RLSA significantly improves performance on DNA sequence-to-graph alignment tasks, particularly for long-read alignment. RLSA outperforms current mainstream heterogeneous scheduling strategies such as HEFT, PEFT, and READYS, demonstrating its superior ability to efficiently handle irregular workloads and adapt to dynamic execution environments.

## ACM Reference Format:

Anonymous Author(s). 2026. A Heterogeneous Computing Architectures for Sequence-to-Graph Alignment. In . ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

## 1 Introduction

With the rapid advancement of high-throughput sequencing and population genomics(Fig.1), DNA alignment has become increasingly constrained by both the expressive power of reference models and the efficiency of computing platforms. Traditional read mapping is primarily formulated as sequence-to-sequence alignment against a single linear reference. While this approach works reasonably well in regions with limited variation, it becomes inadequate in highly polymorphic loci, structurally variable regions, and pangenomic contexts. Linear references cannot simultaneously represent multiple alleles, insertion/deletion events, and larger structural

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).  
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM  
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

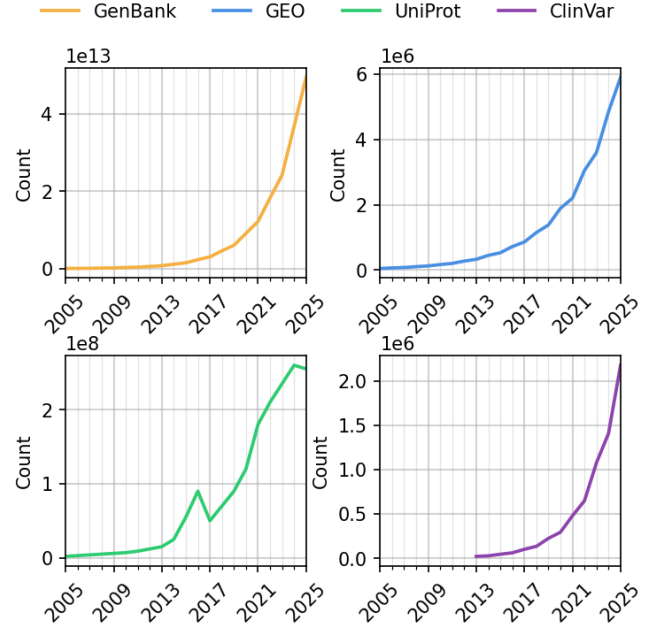


Figure 1: Database growth trends.

rearrangements, often introducing reference bias and causing misaligned reads, distorted scores, and inaccurate downstream interpretation of genomic variation [6, 10, 16].

To overcome these limitations, variation graphs and pangenome graphs have been proposed as richer reference models capable of encoding multiple allelic paths and diverse genomic structures within a unified representation. Graph-based references can explicitly model branching and merging alternatives, reducing reference bias and improving alignment accuracy in polymorphic and structurally complex regions. Consequently, the alignment problem naturally evolves from sequence-to-sequence to sequence-to-graph alignment, where a read must be aligned against one or more candidate paths embedded in a graph.[15, 27, 28, 33].

However, this improved modeling capability comes at a substantial computational cost. Unlike classical dynamic programming on a regular matrix, sequence-to-graph alignment must handle graph branching, merging, and path competition, which generalize dependency structures from fixed neighbors to multiple graph predecessors. The computational cost is highly input-dependent: read length, sequencing error profile, local branching factor, and graph topology jointly determine the size and irregularity of the explored state space. Moreover, the alignment pipeline consists of multiple stages—graph preprocessing, candidate selection, similarity estimation, and local dynamic programming—each with differing parallelism, memory behavior, and control irregularity. As a result, it is

challenging for a single hardware platform to efficiently support the entire workflow [14, 28, 33].

To address these challenges, we propose a heterogeneous architecture for sequence-to-graph DNA alignment. The system first transforms the biological graph into a hardware-friendly sparse intermediate representation, then decomposes the alignment process into fine-grained subtasks connected by dynamic dependencies. An agent-assisted Reinforcement Learning Scheduling Accelerator (RLSA) is employed to adaptively map subtasks to heterogeneous execution domains based on task context, dependency status, and observed execution cost. By combining graph restructuring, dependency-aware task generation, online adaptive scheduling, and feature-matched heterogeneous execution, the proposed design reduces end-to-end latency and improves robustness for irregular alignment workloads.

The main contributions of this work are summarized as follows:

- (1) **Design of a heterogeneous computing architecture:** A collaborative framework targeting DNA sequence-to-graph alignment, addressing task scheduling, graph data representation, and dependency management. The system significantly improves execution efficiency by leveraging an heterogeneous platform containing CPU, GPU, and FPGA resources.
- (2) **Design and hardware implementation of RLSA:** RLSA dynamically updates execution costs and task context during runtime, avoiding pre-assumed execution times. By adapting task scheduling based on real-time learned information, RLSA ensures optimal load balancing across heterogeneous resources, reducing scheduling overhead and improving execution efficiency for irregular workloads.

## 2 Related Work

Prior studies on high-performance DNA alignment can be broadly grouped into three lines of research: acceleration of sequence-to-sequence alignment on linear references, algorithmic advances in sequence-to-graph alignment, and recent efforts on parallel or heterogeneous acceleration for graph-based alignment. Since the algorithmic formulation of sequence-to-graph alignment is introduced in the next section, this subsection emphasizes architectural, hardware, and scheduling aspects rather than the recurrence equations themselves.

A large body of prior work has targeted classical sequence-to-sequence alignment, especially Smith–Waterman and its variants. Representative approaches include SIMD vectorization [8], GPU acceleration [18], FPGA systolic arrays [9], and cluster-scale heterogeneous systems [20]. These studies demonstrate that substantial speedup can be obtained when the workload exposes regular dynamic-programming dependencies and relatively predictable memory access patterns. However, these methods are fundamentally designed for linear-reference alignment, where the dependency structure is fixed and the execution pattern is comparatively uniform. As a result, their optimization principles do not directly transfer to sequence-to-graph alignment, where branching, merging, and path competition induce irregular control flow, sparse connectivity, and input-dependent workload variation.

The transition from linear references to variation graphs or pangenome graphs is motivated by the need to represent multiple alleles, insertions/deletions, and larger structural variants in a unified reference model, thereby reducing reference bias and improving alignment accuracy in highly polymorphic genomic regions [6, 10, 16]. However, this increased biological expressiveness comes at a substantial systems cost. In sequence-to-graph alignment, a read is no longer aligned against a single regular matrix induced by one reference string; instead, dynamic programming must propagate across graph topologies with multiple predecessors and competing candidate paths [15, 27, 28]. Consequently, the computational cost depends not only on read length, but also on graph branching factor, local topology, and sequencing error characteristics. Compared with conventional sequence alignment, the workload is therefore more irregular in three dimensions: dependency structure, memory access behavior, and runtime execution cost. This makes sequence-to-graph alignment substantially harder to optimize using a single software strategy or a single hardware target.

A second line of research has improved sequence-to-graph alignment primarily from the algorithmic perspective. Early work on partial-order alignment established the dynamic-programming foundation for aligning sequences to graph-like references [15]. More recent studies introduced bit-parallel techniques, graph-aware banding, and wavefront-style acceleration to reduce the explored state space while preserving alignment quality [27, 28, 33]. These methods significantly improved the practicality of graph-based alignment and showed that exact or near-exact alignment to graphs can be made much more efficient than naive formulations. Nevertheless, most of these efforts remain predominantly CPU-centric and focus on algorithmic pruning, state compression, or search-space reduction. They do not explicitly address how different phases of the alignment pipeline should be co-designed with hardware, nor do they provide a runtime mechanism for adapting execution to changing workload characteristics.

A smaller but growing body of work has started to consider parallelization and systems optimization for graph-based alignment. Jain *et al.* [14] showed that sequence-to-graph alignment can benefit from parallel execution and careful system-level implementation. More recent graph wavefront methods also suggest that graph alignment can expose useful parallelism when computation is reorganized appropriately [33]. However, most existing solutions target only a specific kernel, data structure, or optimization stage. They generally do not cover the entire end-to-end pipeline, including graph preprocessing, task generation, dependency management, cross-device execution, and runtime adaptation.

Sequence-to-graph alignment is not merely a harder version of sequence-to-sequence alignment; it is a *phase-heterogeneous* workload. Different stages of the pipeline exhibit markedly different computational characteristics. For example, graph preprocessing and candidate filtering often favor throughput-oriented parallel processing, while local graph-aware dynamic programming is dominated by fine-grained dependencies and irregular state transitions. This mismatch implies that no single hardware platform is ideal for all stages. GPUs are well suited to massively parallel, throughput-dominated subtasks; FPGAs are attractive for deeply pipelined, dependency-sensitive kernels; and CPUs remain useful

for control-intensive orchestration and irregular runtime management [9, 19, 20]. Therefore, heterogeneous computing is not only beneficial for acceleration, but also structurally aligned with the intrinsic heterogeneity of the sequence-to-graph alignment pipeline.

Despite the above progress, three important limitations remain. First, most prior work does not treat sequence-to-graph alignment as a full-stack co-design problem spanning graph representation, task decomposition, scheduling, and cross-device execution. Second, device assignment is typically static or heuristic, even though the cost of graph alignment varies strongly with read length, local graph complexity, and dependency density. Third, online scheduling mechanisms that combine task context with runtime feedback are still largely missing. These limitations are especially harmful for DNA sequence-to-graph alignment, where the same input batch may contain both highly regular and highly irregular subtasks. In such settings, static mapping can easily lead to load imbalance, poor resource utilization, and excessive scheduling overhead.

Our work is motivated by the observation that high-performance DNA sequence-to-graph alignment requires *software-hardware co-design* rather than isolated algorithmic or hardware optimizations. At the software level, graph restructuring and dependency-aware task decomposition are necessary to expose executable units from an irregular biological graph. At the hardware level, heterogeneous execution allows different subtasks to be matched to the most appropriate compute domain. At the runtime level, adaptive scheduling is required to respond to input-dependent variability and dynamic device conditions. In this sense, our work differs from prior studies by targeting an end-to-end heterogeneous framework that jointly addresses graph-friendly data representation, dynamic dependency management, and online hardware-aware task scheduling.

### 3 Sequence-to-Graph Alignment

#### 3.1 Graph Model and Problem Statement

Graph-based reference sequences compactly encode alleles, haplotypes, and larger structural variants, thereby reducing reference bias and improving the representation of highly polymorphic loci and pangenomic diversity [6, 10, 16]. In this work, we model the reference as a *directed acyclic graph* (DAG)  $G = (V, E, \sigma)$ , where each vertex  $v \in V$  carries a character label  $\sigma(v) \in \Sigma$  (e.g.,  $\Sigma = \{A, C, G, T\}$  for DNA), and each directed edge  $(u, v) \in E$  permits the concatenation of  $\sigma(u)$  followed by  $\sigma(v)$  [15, 27].

Any directed path  $p = (v_{t_1}, \dots, v_{t_\ell})$  spells a string

$$\sigma(p) = \sigma(v_{t_1}) \cdots \sigma(v_{t_\ell}).$$

Let the query sequence be  $q = q_1 q_2 \cdots q_m \in \Sigma^*$  with length  $m$ . We study *local* sequence-to-graph alignment, where the optimal alignment may start and end at arbitrary positions in the query and at arbitrary locations along a graph path, analogous to Smith-Waterman local alignment on strings [14, 29].

**Definition 3.1 (Sequence-to-DAG Local Alignment).** Given a query sequence  $q$  and a labeled DAG  $G = (V, E, \sigma)$ , find a path  $p$  in  $G$  and a substring  $q[a..b]$  such that the optimal alignment score between  $q[a..b]$  and  $\sigma(p)$  is maximized over all choices of  $p, a, b$ , and output one corresponding optimal alignment.

#### 3.2 Scoring Scheme

We adopt an additive scoring scheme consisting of (i) a substitution score for aligning query character  $q_i$  to vertex  $v_j$ , and (ii) linear gap penalties. The substitution score is defined as

$$\Delta_{i,j} \triangleq \begin{cases} +\mu, & q_i = \sigma(v_j), \\ -\nu, & q_i \neq \sigma(v_j), \end{cases} \quad \mu > 0, \nu > 0, \quad (1)$$

and let  $\Delta_{\text{ins}} > 0$  and  $\Delta_{\text{del}} > 0$  denote the gap penalties.

To avoid ambiguity, we use the following operational interpretation: (a) *gap in the query* means advancing in the graph without consuming a query symbol (penalty  $\Delta_{\text{ins}}$ ); (b) *gap in the graph* means consuming a query symbol without advancing in the graph (penalty  $\Delta_{\text{del}}$ ). Compared with affine-gap formulations, linear penalties keep the recurrence compact and are widely used as a baseline in exact sequence-to-graph alignment and its high-performance implementations [11, 22].

#### 3.3 Dynamic Programming on a DAG

Assume the vertices are arranged in a topological order  $v_1, \dots, v_{|V|}$ . Let

$$\text{pred}(j) \triangleq \{k \mid (v_k, v_j) \in E\} \quad (2)$$

denote the set of in-neighbors of vertex  $v_j$ . We define the local-alignment DP state as

$$C_{i,j} = \max_{\substack{1 \leq k \leq i \\ p \rightsquigarrow v_j}} \text{score}(q[k..i], \sigma(p)), \quad (3)$$

where  $p \rightsquigarrow v_j$  denotes any path in  $G$  ending at  $v_j$ , and the maximization over  $k$  allows the alignment to begin at any query position. As in Smith-Waterman local alignment, negative scores are clamped to zero so that the alignment may restart anywhere. We initialize

$$C_{0,j} = 0, \quad \forall j \in \{1, \dots, |V|\}. \quad (4)$$

For  $i \in \{1, \dots, m\}$  and  $j \in \{1, \dots, |V|\}$ , the local-alignment recurrence is

$$C_{i,j} = \max \begin{cases} 0 & \text{(restart)} \\ \Delta_{i,j} & \text{(start at } (i, j)) \\ \max_{k \in \text{pred}(j)} (C_{i-1,k} + \Delta_{i,j}) & \text{(match/mismatch via edge)} \\ \max_{k \in \text{pred}(j)} (C_{i,k} - \Delta_{\text{ins}}) & \text{(gap in query)} \\ C_{i-1,j} - \Delta_{\text{del}} & \text{(gap in graph)} \end{cases} \quad (5)$$

where, if  $\text{pred}(j) = \emptyset$ , the corresponding maxima are treated as  $-\infty$  and therefore do not contribute.

Relative to classical string-to-string Smith-Waterman, the key structural change is that the diagonal dependency is generalized from a single predecessor cell to all in-neighbors of  $v_j$  in the DAG. This is the standard dynamic-programming view underlying partial-order alignment and exact sequence-to-graph alignment on DAGs [22, 27].

Evaluating (5) over all  $(i, j)$  yields a  $O(m(|V| + |E|))$  time complexity, since the in-neighbor maximizations can be aggregated for each query layer. A row-wise implementation stores only the current and previous query layers during score computation, resulting in  $O(|V|)$  working memory for scoring alone [14, 22]. To recover one optimal alignment, we store backpointers for the maximizing

transition and trace back from the highest-scoring cell until a cell with value 0 is reached, exactly as in Smith–Waterman local alignment. In practical graph aligners, this exact DP formulation is often combined with indexing, seed-and-extend, banding, or wavefront-style acceleration to reduce the explored DP region on large graphs [10, 28, 33].

## 4 System Description

To address the challenges of DNA sequence map alignment in data representation, task organization, heterogeneous scheduling, and execution acceleration, this paper designs a collaborative acceleration system suitable for heterogeneous platforms, as shown in Fig 3. This chapter designs the system from four levels: graph data preprocessing, dynamic task generation and dependency control, reinforcement learning scheduling, and heterogeneous execution acceleration, constructing a complete processing chain of data structure reconstruction—task graph construction—online scheduling decision—heterogeneous execution feedback.

### 4.1 Graph Structuring and Sparse Transformation

In DNA sequence-to-graph alignment, a primary challenge lies not only in the computation itself but also in the organization of the input reference graph. Unlike linear references, variation graphs represent genomic variations by introducing branching and merging topologies, which significantly increase structural irregularity during traversal and alignment [10, 27]. Although node attributes are relatively regular, typically consisting of nucleotide symbols and limited auxiliary topology information, edge connectivity is highly irregular, especially in variation-dense regions. As a result, directly storing and processing the original graph with adjacency lists or pointer-based structures leads to redundant storage, poor memory locality, and weak compatibility with parallel hardware execution.

To address this issue, the proposed system performs graph structuring and sparse transformation on the host side before heterogeneous execution. The objective of this stage is not merely to reduce storage overhead, but to convert the original biologically oriented graph representation into a hardware-friendly intermediate form suitable for subsequent task generation and scheduling. Specifically, each nucleotide symbol in the DNA graph is encoded using a compact 2-bit representation, thereby reducing memory consumption and host-device transfer overhead. On top of this encoding, each graph node is further transformed into a unified intermediate representation (IR), which integrates both content and execution-related metadata, including topology flags for branching/merging nodes, sparse row offsets, node degree information, and local structural complexity descriptors. Through this transformation, the original biological graph is converted into a set of addressable, schedulable, and statistically analyzable computation nodes.

After graph structuring, the reference graph is further mapped into a compressed sparse row (CSR) format, which is widely adopted for sparse data representation due to its compact storage and efficient row-wise access characteristics. This choice is motivated by the core access pattern of sequence-to-graph alignment, where

all outgoing neighbors of a current node must be enumerated efficiently for path extension or local state transition. Concretely, the transformed graph is represented by arrays `row_ptr` and `col_idx`, together with optional auxiliary arrays for edge labels or weights. For a node  $v_i$ , its outgoing edges can be accessed directly from the contiguous interval `row_ptr[i]` to `row_ptr[i+1]-1`. Compared with pointer-based adjacency lists, CSR eliminates frequent pointer dereferencing and irregular memory access. Compared with coordinate-style sparse formats, CSR better matches node-centric forward expansion and is more suitable for coalesced memory access on GPUs as well as streaming-style processing on FPGAs.

### 4.2 Parallelization Engine Based on Dynamic Computation Graph Dependencies

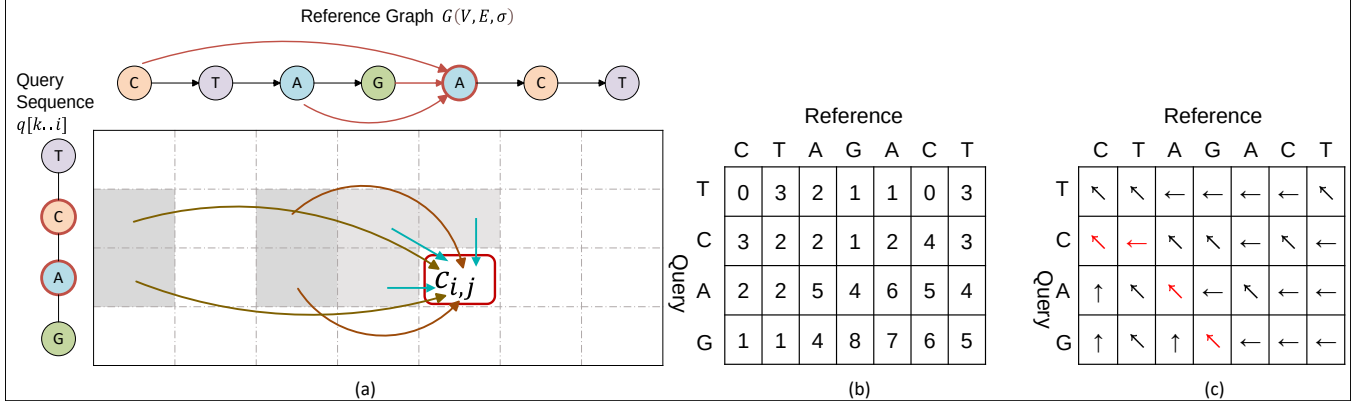
After graph structuring, we develop a dynamic-computation-graph-based parallelization engine to capture the irregular and runtime-varying dependencies in DNA sequence-to-graph alignment. Inspired by dynamic execution in deep learning frameworks and dependency-driven heterogeneous runtimes, the alignment procedure is decomposed into fine-grained operators whose dependencies are instantiated online according to the current dataflow [4, 7, 26].

This design is motivated by the characteristics of sequence-to-graph alignment. Owing to graph branching/merging structures, read-specific errors, and local topology variation, task complexity and dependency patterns vary significantly across reads and graph regions. Therefore, static partitioning is insufficient to maintain both load balance and execution efficiency. In this work, tasks are partitioned along local traversal and state-transition dependencies so that each subtask preserves dependency locality while exposing parallelism.

To support low-overhead execution, we propose a hardware-coupled Task Partitioning and Dependency Controller (TPDC), which parses the sparse graph stream online and maps it to pre-defined operators, thereby generating executable subtasks with reduced software overhead. Dynamic Dependency Tracking Registers (DDTRs) are introduced to maintain unresolved predecessor counts, and a subtask is released only after all its dependencies are satisfied. This enables asynchronous dataflow-style execution and reduces pipeline stalls. Meanwhile, to address workload imbalance, TPDC adopts an adaptive binning strategy that groups subtasks according to computational density and local graph complexity, and dispatches them to the most suitable execution domain. In general, massively parallel subtasks are assigned to GPUs, whereas regular and pipeline-friendly subtasks are offloaded to FPGAs. Through this co-designed mechanism, the proposed engine reduces dynamic scheduling overhead and improves parallel efficiency under irregular dependency constraints.

### 4.3 RL-Based Scheduling Engine

Since the computational cost of DNA sequence-to-graph alignment is highly dependent on input statistics, rule-based or heuristic schedulers are often insufficient to achieve stable load balancing across heterogeneous devices. To address this issue, we employ a reinforcement learning-based task scheduling algorithm that learns online the execution preferences of different hardware targets and



**Figure 2: Illustration of sequence-to-graph local alignment. (a) An example alignment between a query and a DAG path. (b) The corresponding DP score matrix. (c) The traceback matrix. Both matrices use  $\mu = 3$ ,  $\nu = 1$ ,  $\Delta_{\text{ins}} = 1$ , and  $\Delta_{\text{del}} = 1$ .**

the execution times of different tasks, thereby making fast and accurate scheduling decisions[5, 21, 30].

To support efficient scheduling algorithms based on reinforcement learning, we further designed a Reinforcement Learning Scheduling Accelerator (RLSA), which hardwareizes policy inference and reward updates. Its key component is the Context Cost Matrix (CCM), which stores the historical correlation between task features and observed device execution costs. The CCM is necessary because scheduling decisions are triggered at subtask granularity, while runtime cost statistics must be queried and updated concurrently; otherwise, the scheduler would rely on software-managed profiling or off-chip accesses, making decision latency a system bottleneck. Based on the CCM, the feature-aware agent core extracts task context and generates a hardware mapping decision using a hardwareized  $\epsilon$ -greedy policy, while the feedback unit converts measured execution latency into reward signals for online update. The reinforcement learning scheduling algorithm based on this structure is shown in Algorithm 1.

This algorithm-architecture co-design significantly reduces scheduling overhead and enables the scheduler to continuously adapt to input variation and hardware load fluctuation. As a result, the proposed engine improves both long-term performance and robustness for irregular DNA sequence-to-graph alignment workloads [21].

#### 4.4 Efficient Multidimensional Acceleration for Heterogeneous Systems

After scheduling, subtasks are dispatched to heterogeneous execution domains for actual computation. We observe that DNA sequence-to-graph alignment exhibits strong phase heterogeneity during execution. In particular, the embedding and similarity estimation stages based on the Omni-DNA model[17] are dominated by dense linear algebra and data-parallel operations, whereas the subsequent local alignment stage is constrained by fine-grained data dependencies and latency-sensitive state transitions [9, 19]. Therefore, we adopt a feature-matched execution strategy that maps subtasks to the most suitable hardware domain according to their computational characteristics.

#### Algorithm 1 RL-Based Hardware-Aware Adaptive Scheduling

**Require:** Ready subtask queue  $Q$ , device set  $\mathcal{D}$ , context-cost matrix CCM, exploration rate  $\epsilon$ , learning rate  $\alpha$

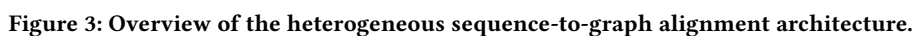
**Ensure:** Hardware mapping decision for each ready subtask

```

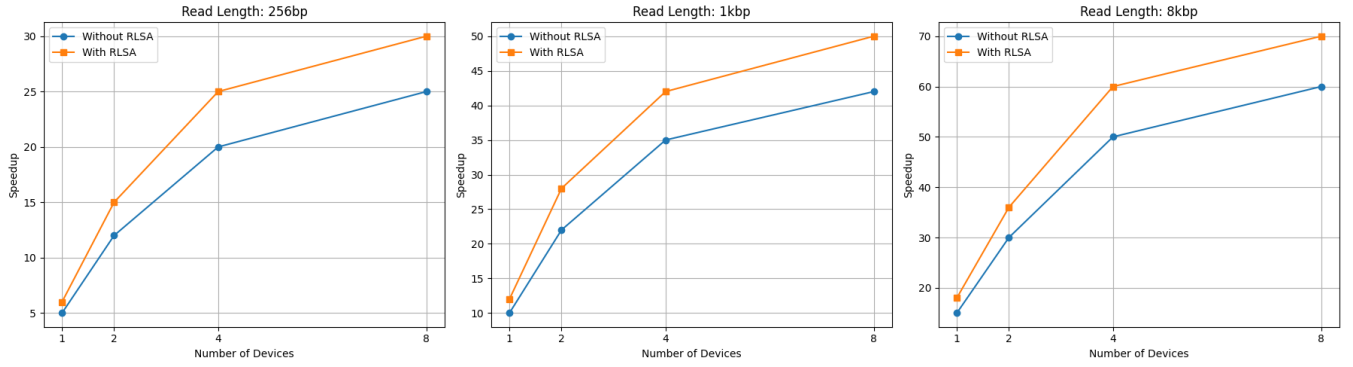
1: while  $Q \neq \emptyset$  do
2:   Pop a ready subtask  $\tau_i$  from  $Q$ 
3:   Extract context feature vector  $x_i \leftarrow \Phi(\tau_i)$ 
4:   for all  $d \in \mathcal{D}$  do
5:     Query estimated execution cost  $\hat{c}_{i,d} \leftarrow \text{CCM}(x_i, d)$ 
6:   end for
7:   Generate a random number  $u \sim U(0, 1)$ 
8:   if  $u < \epsilon$  then
9:     Select device  $d_i$  randomly from  $\mathcal{D}$ 
10:  else
11:    Select device  $d_i \leftarrow \arg \min_{d \in \mathcal{D}} \hat{c}_{i,d}$ 
12:  end if
13:  Dispatch  $\tau_i$  to device  $d_i$ 
14:  Measure actual execution latency  $c_{i,d_i}$ 
15:  Compute reward  $r_i \leftarrow -c_{i,d_i}$ 
16:  Update CCM entry:
       $\text{CCM}(x_i, d_i) \leftarrow (1 - \alpha) \text{CCM}(x_i, d_i) + \alpha c_{i,d_i}$ 
17:  for all successor task  $\tau_j$  of  $\tau_i$  in DAG do
18:    Decrease unresolved dependency counter of  $\tau_j$ 
19:    if all predecessors of  $\tau_j$  are completed then
20:      Push  $\tau_j$  into  $Q$ 
21:    end if
22:  end for
23: end while

```

At the architectural level, a device-level scheduler manages a pool of specialized accelerators. For data-parallel subtasks, we employ a GPU-based vector compute engine, which exploits massive thread-level parallelism and tensorized matrix-vector execution to accelerate embedding and similarity computation. For dependency-dominated subtasks, we use an FPGA-based alignment systolic array, where the dynamic programming procedure is spatially mapped onto pipelined processing elements to enable streaming execution

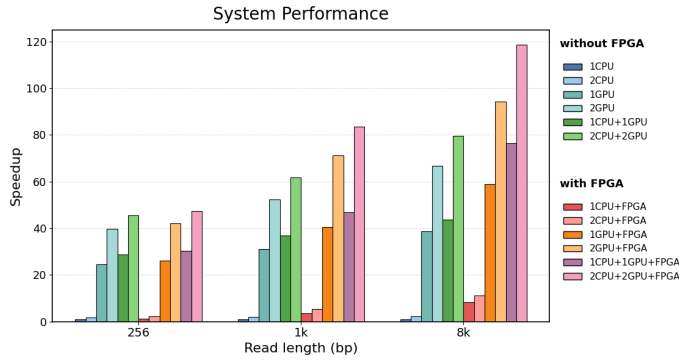


To support online optimization, each execution domain is equipped with lightweight performance monitors that record runtime latency



**Figure 4: Speedup comparison between the alignment algorithm with and without RLSA under different read lengths and numbers of devices.**

and send feedback to the scheduling engine through a dedicated control path. This hardware-level closed loop allows the scheduler to continuously adapt task mapping decisions to workload variation and device status changes, thereby improving long-term performance and resource utilization.



**Figure 5: System performance under different heterogeneous configurations.**

## 5 HARDWARE AND SOFTWARE PLATFORMS

The proposed system is implemented on a heterogeneous platform that includes AMD EPYC 9654 CPUs, NVIDIA H100 GPUs, and AMD Xilinx Alveo U280 FPGAs [1, 2, 24]. To facilitate real-time sequencing, the pipeline is designed to accept streaming reads from the Oxford Nanopore MinION Mk1D device [25].

For graph-based alignment, the reference graph is constructed using data from the Human Pangenome Reference Consortium (HPRC), which offers a comprehensive representation of genomic diversity, extending beyond a single linear reference [13]. In terms of sequence queries and validation, we rely on the NCBI RefSeq database and UniProtKB/Swiss-Prot, both of which provide high-quality, meticulously curated genomic and protein sequences [23, 32].

The software environment is built upon Ubuntu Linux, with GPU acceleration supported by CUDA, and FPGA development

facilitated through Xilinx Vitis. The sequence-to-graph alignment process is carried out using our proposed DAG-based engine, which integrates runtime scheduling across multiple heterogeneous computing devices for optimized performance.

## 6 PERFORMANCE EVALUATION

### 6.1 RLSA Performance Analysis

To evaluate the effectiveness of the RLSA, we conducted experiments under varying read lengths (256 bp, 1 kbp, and 8 kbp) and numbers of computing devices. These read lengths were sampled from sequences in the NCBI RefSeq database and UniProtKB/Swiss-Prot, all derived from the GRCh38 human genome assembly on chromosome 7. Specifically, the 256 bp, 1 kbp, and 8 kbp reads were extracted from the region spanning positions 116,500,000 to 119,500,000 on chromosome 7. For each read length, sequences were randomly sampled and cropped into smaller segments corresponding to the specified read lengths (256 bp, 1 kbp, and 8 kbp). The total number of reads was 5000 for the 256 bp category, 2000 for the 1 kbp category, and 200 for the 8 kbp category, with each set ensuring that the sequence overlap did not exceed 5%. The variation graph was sourced from the HPRC and the reference genome was the GRCh38 human genome assembly, specifically focusing on chromosome 7. Figure 4 shows the speedup achieved by the alignment algorithm with and without RLSA.

The main reason is that RLSA does not rely on a fixed execution-time assumption. Instead, it queries and updates the CCM online, so that the scheduler can estimate the current execution cost of a subtask on CPU, GPU, and FPGA according to its context and recent observations. Therefore, the mapping decision can be adjusted at runtime rather than being determined only by static priorities or offline estimates.

This mechanism is especially effective for long-read scheduling. When a long-read alignment task arrives, if the FPGA is idle, RLSA directly dispatches it to the FPGA, which is well suited for dependency-sensitive local alignment. If the FPGA is busy, RLSA compares the waiting time for the current FPGA task to finish plus the predicted FPGA processing time against the predicted heterogeneous CPU/GPU execution time. If the former is smaller, the task waits for the FPGA; otherwise, it is reassigned to CPU/GPU

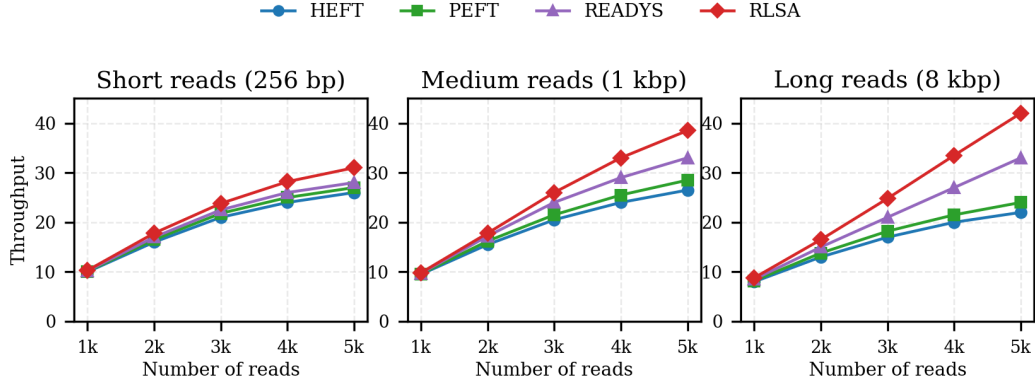


Figure 6: Throughput comparison of HEFT, PEFT, READYS, and RLSA under different read lengths.

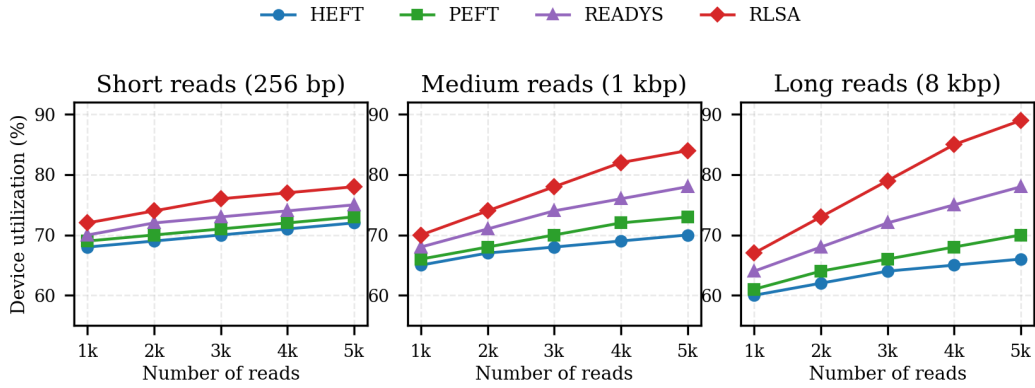


Figure 7: Device utilization comparison of HEFT, PEFT, READYS, and RLSA under different read lengths.

resources. Since this comparison is obtained by table lookup from CCM, its overhead is negligible relative to task execution time. Similarly, CPUs are preferred for control-intensive and irregular subtasks, whereas GPUs are favored for massively parallel similarity computation. As a result, RLSA reduces idle time and pipeline stalls, and its advantage becomes more pronounced as the number of devices increases.

## 6.2 System Performance with RLSA Optimization

To evaluate the overall performance of the proposed heterogeneous architecture, we first compare different hardware configurations across varying read lengths, as shown in Fig. 5. The results show that heterogeneous execution significantly improves performance over the CPU-only baseline, and the speedup increases as more accelerators are introduced, reaching up to 118.7 $\times$  for long reads under the 2CPU+2GPU+1FPGA configuration.

The performance gain strongly depends on read length. For short reads, GPU-oriented configurations provide substantial benefit because the workload is more data parallel, whereas the benefit of FPGA is relatively limited. As the read length increases to 1 kbp and 8 kbp, FPGA-related configurations become increasingly effective, reflecting the growing importance of dependency-sensitive dynamic programming. Hybrid CPU/GPU/FPGA configurations therefore achieve the best overall speedup by combining the complementary strengths of different hardware resources.

Figures 6 and 7 further compare HEFT [31], PEFT [3], READYS [12], and the proposed RLSA. For short reads, the gap among schedulers is small because static or semi-static estimates are already adequate for relatively regular workloads. However, for medium and especially long reads, RLSA achieves the highest throughput and device utilization. The reason is that HEFT-, PEFT-, and READYS-style policies mainly depend on pre-estimated costs or fixed priorities, and thus cannot react effectively once runtime conditions deviate from the assumed model. In contrast, RLSA continuously updates CCM with observed latency and uses the updated cost to decide

whether a task should wait for FPGA execution or be remapped to CPU/GPU resources. This online comparison improves both FPGA assignment for dependency-dominated subtasks and CPU/GPU assignment for control-intensive or data-parallel stages, thereby reducing suboptimal mappings and resource under-utilization.

These results demonstrate that high performance in sequence-to-graph alignment depends on both heterogeneous hardware integration and online adaptive scheduling. By combining multi-device acceleration with CCM-supported runtime learning, the proposed system effectively handles irregular workloads and scales better with increasing read length and workload size.

## 7 Conclusions

In this work, we proposed a heterogeneous architecture for sequence-to-graph DNA alignment, addressing key challenges such as irregular workloads, multi-stage pipelines, and dynamic computational costs. We introduced a hardware-assisted RLSA that adaptively maps fine-grained subtasks to heterogeneous devices, optimizing task execution. Our experimental results demonstrate that the proposed system significantly enhances speedup and robustness, particularly for medium and long reads, compared to baseline methods. By integrating graph restructuring, dependency-aware task decomposition, and RL-based adaptive scheduling, our approach offers an effective solution for high-performance, real-time sequence-to-graph alignment.

Despite the promising results, there are several areas for future research. One limitation of the current system is the dependency on specific hardware platforms, which may not be universally available or suitable for all use cases. Future work could focus on optimizing the architecture for a broader range of hardware configurations to increase accessibility. Additionally, while the proposed system effectively handles medium and long reads, there is still room to improve its performance for short reads and highly variable input data. Exploring more advanced reinforcement learning techniques and further refining the task scheduling process could contribute to even greater improvements in alignment accuracy and efficiency. Lastly, the scalability of the system across larger genomic datasets remains an open challenge, warranting further exploration in future research.

## References

- [1] AMD. Alveo U280 Accelerator Card. <https://www.amd.com/zh-cn/support/downloads/alveo-downloads.html/accelerators/alveo/u280.html>, 2025. Last accessed 2025/04/09.
- [2] AMD. AMD EPYC 9654. <https://www.amd.com/en/products/processors/server/epyc/4th-generation-9004-and-8004-series/amd-epyc-9654.html>, 2025. Last accessed 2025/04/09.
- [3] ARABNEJAD, H., AND BARBOSA, J. G. List scheduling algorithm for heterogeneous systems by an optimistic cost table. *IEEE Transactions on Parallel and Distributed Systems* 25, 3 (2014), 682–694.
- [4] AUGONNET, C., THIBAUT, S., NAMYST, R., AND WACRENIER, P.-A. StarPU: A unified platform for task scheduling on heterogeneous multicore architectures. In *Euro-Par 2009 Parallel Processing* (2009), vol. 5704 of *Lecture Notes in Computer Science*, Springer, Heidelberg, pp. 863–874.
- [5] CHU, W., LI, L., REYZIN, L., AND SCHAPIRE, R. E. Contextual bandits with linear payoff functions. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)* (2011), vol. 15 of *Proceedings of Machine Learning Research*, pp. 208–214.
- [6] DILTNEY, A., COX, C., IQBAL, Z., NELSON, M. R., ET AL. Improved genome inference in the MHC using a population reference graph. *Nature Genetics* 47, 6 (2015), 682–688.
- [7] DURAN, A., AYGUADÉ, E., BADIA, R. M., LABARTA, J., MARTINELL, L., MARTORELL, X., AND PLANAS, J. OmpSs: A proposal for programming heterogeneous multi-core architectures. *Parallel Processing Letters* 21, 2 (2011), 173–193.
- [8] FARRAR, M. Striped smith–waterman speeds database searches six times over other SIMD implementations. *Bioinformatics* 23, 2 (2007), 156–161.
- [9] FEI, X., DAN, Z., LINA, L., XIN, M., AND CHUNLEI, Z. FPGASW: Accelerating large-scale smith–waterman sequence alignment application with backtracking on FPGA linear systolic array. *Interdisciplinary Sciences: Computational Life Sciences* 10, 1 (2018), 176–188.
- [10] GARRISON, E., SIRÉN, J., NOVAK, A. M., HICKEY, G., EIZENGA, J. M., DAWSON, E. T., JONES, W., GARG, S., MARKELLO, C., LIN, M. F., PATEN, B., AND DURBIN, R. Variation graph toolkit improves read mapping by representing genetic variation in the reference. *Nature Biotechnology* 36, 9 (2018), 875–879.
- [11] GOTOH, O. An improved algorithm for matching biological sequences. *Journal of Molecular Biology* 162, 3 (1982), 705–708.
- [12] GRINSZTAJN, N., BEAUMONT, O., JEANNOT, E., AND PREUX, P. Readys: A reinforcement learning based strategy for heterogeneous dynamic scheduling. In *2021 IEEE International Conference on Cluster Computing (CLUSTER)* (2021), pp. 192–203.
- [13] HUMAN PANGENOME REFERENCE CONSORTIUM. Human Pangenome Reference Consortium. <https://humanpangenome.org/>, 2025. Last accessed 2025/04/09.
- [14] JAIN, C., MISRA, S., ZHANG, H., DILTNEY, A., AND ALURU, S. Accelerating sequence alignment to graphs. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)* (2019), IEEE, pp. 451–461.
- [15] LEE, C., GRASSO, C., AND SHARLOW, M. F. Multiple sequence alignment using partial order graphs. *Bioinformatics* 18, 3 (2002), 452–464.
- [16] LI, H., FENG, X., AND CHU, C. The design and construction of reference pangenome graphs with minigraph. *Genome Biology* 21 (2020), 265.
- [17] LI, Z., SUBASRI, V., SHEN, Y., LI, D., ZHAO, Y., STAN, G.-B., AND SHAN, C. Omni-dna: A unified genomic foundation model for cross-modal and multi-task learning. *arXiv preprint arXiv:2502.03499* (2025).
- [18] MANAVSKI, S. A., AND VALLE, G. CUDA compatible GPU cards as efficient hardware accelerators for smith–waterman sequence alignment. *BMC Bioinformatics* 9, Suppl 2 (2008), S10.
- [19] MARKIDIS, S., CHIEN, S. W. D., LAURE, E., PENG, I. B., AND VETTER, J. S. NVIDIA tensor core programmability, performance & precision. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)* (2018), pp. 522–531.
- [20] MENG, X., AND CHAUDHARY, V. A high-performance heterogeneous computing platform for biological sequence analysis. *IEEE Transactions on Parallel and Distributed Systems* 21, 9 (2010), 1267–1280.
- [21] NARAYANAN, D., SANTHANAM, K., KAZHAMIAKA, F., PHANISHAYEE, A., AND ZAHARIA, M. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)* (2020), pp. 481–498.
- [22] NAVARRO, G. Improved approximate pattern matching on hypertext. *Theoretical Computer Science* 237, 1–2 (2000), 455–463.
- [23] NCBI. RefSeq: NCBI Reference Sequence Database. <https://www.ncbi.nlm.nih.gov/refseq/>, 2025. Last accessed 2025/04/09.
- [24] NVIDIA. NVIDIA H100 Tensor Core GPU. <https://www.nvidia.com/en-us/data-center/h100/>, 2025. Last accessed 2025/04/09.
- [25] OXFORD NANOPORE TECHNOLOGIES. MinION Mk1D device and IT specifications. <https://nanoporetech.com/zh/document/requirements/minion-mk1d-device-and-it-specifications>, 2025. Last accessed 2025/04/09.
- [26] PASZKE, A., GROSS, S., MASSA, F., LERER, A., BRADBURY, J., CHANAN, G., KILLEEN, T., LIN, Z., GIMELSHEIN, N., ANTIGA, L., DESMAISON, A., KÖPF, A., YANG, E. Z., DEVITO, Z., RAISON, M., TEJANI, A., CHILAMKURTHY, S., STEINER, B., FANG, L., BAI, J., AND CHINTALA, S. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* 32 (2019), pp. 8024–8035.
- [27] RAUTIAINEN, M., MÄKINEN, V., AND MARSCHALL, T. Bit-parallel sequence-to-graph alignment. *Bioinformatics* 35, 19 (2019), 3599–3607.
- [28] RAUTIAINEN, M., AND MARSCHALL, T. Graphaligner: Rapid and versatile sequence-to-graph alignment. *Genome Biology* 21, 1 (2020), 253.
- [29] SMITH, T. F., AND WATERMAN, M. S. Identification of common molecular subsequences. *Journal of Molecular Biology* 147, 1 (1981), 195–197.
- [30] SUTTON, R. S., AND BARTO, A. G. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, Cambridge, MA, 2018.
- [31] TOPCUOĞLU, H., HARIRI, S., AND WU, M.-Y. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems* 13, 3 (2002), 260–274.
- [32] UNIPROT CONSORTIUM. UniProtKB/Swiss-Prot. <https://www.uniprot.org/uniprotkb>, 2025. Last accessed 2025/04/09.
- [33] ZHANG, H., WU, S., ALURU, S., AND LI, H. Fast sequence to graph alignment using the graph wavefront algorithm. *arXiv preprint arXiv:2206.13574* (2022).